AESWARYA KRISHNAKUMAR

INTRODUCTION

Visualising hierarchical and relational data plays a crucial role in the discovery of hidden structures, understanding relationships between elements, and making the comprehension of complicated data easier. An interactive and well-structured visual representations significantly improve comprehension and navigation in complex systems [1]. In this analysis, I explore four distinct visualisation techniques to represent the same hierarchical datase, which is Node-Link Tree Layout, Radial Tree Layout, Treemap Visualisation, and Space-Optimised Hierarchical Graph Visualisation. With all these ways, there are ways of presenting parent child relationships and structural depth of data in different ways. The selected set of data represents a multi-levelled file organization, that is folders and files are inside each other, which enables a detailed comparison of these visual methods. Some Studies emphasise how visual layouts influence user focus and cognitive load when navigating hierarchies [1]. With the help of different methods I intend to demonstrate how layout style affects in the rendering of structure, density and grouping of hierarchical information as proved by Lambert et al with regards to determining the effects of design options in clarity and interpretability.

TOOLS AND TECHNOLOGIES

All four visualisations were developed using Java as the main platform, with the help of DC Treemap, DOITree, and Space-Optimisation Tree Visualisation tools. These tools include inbuilt hierarchical datasets for implementation. The resulting visualisations were exported or saved in the following formats based on the tool used:

- DC Treemap: .jar for the application
- DOITree: .jar for the tool
- Space-Optimisation Tree Visualisation: typically runs as a Java Applet or HTML file like t20.html

VISUALISATIONS

Node-Link Tree Layout

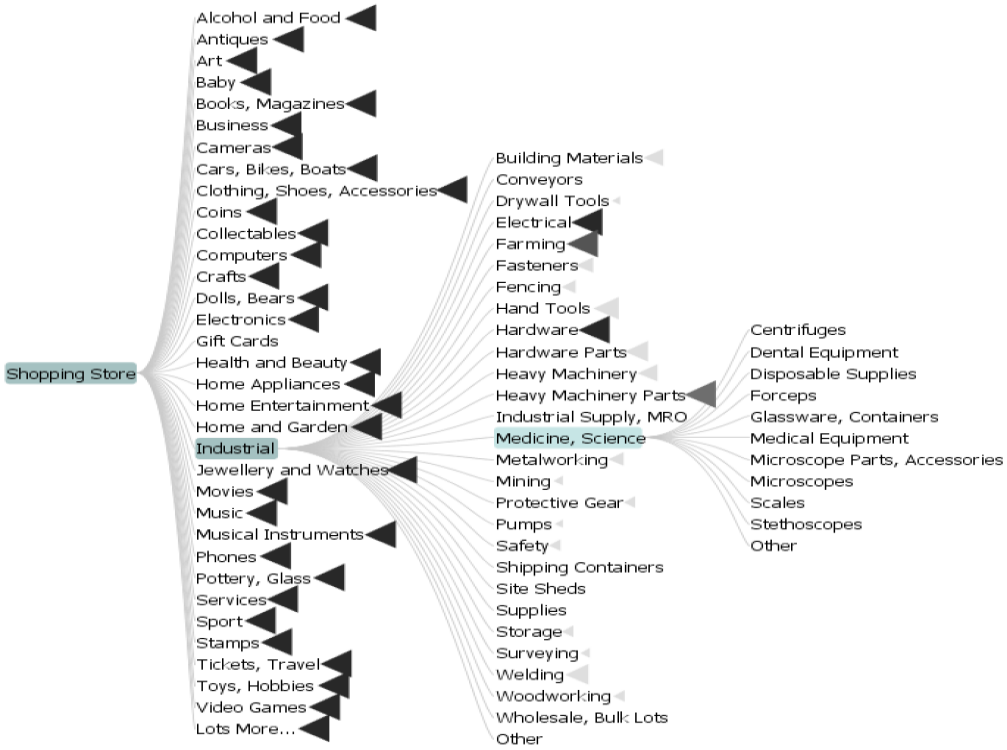

Figure 1

This visualisation presents a hierarchical structure using a Node-Link Tree layout, constructed from the dataset ProductCategoryAustralia_6254.xml. The main node, titled "Shopping Store", represents the top-level category branching all others beneath it. Each line from a parent node to a child node visually expresses a parent-child relationship which is helping to interpret the actual structure of the dataset easily.

The dataset branches out into numerous main product categories such as Books, Electronics, Industrial, Health and Beauty, Jewellery, and more. Each of these main categories can further split into multiple subcategories. For example, the Industrial category expands into areas like Building Materials, Hardware, Medicine, Science, and Heavy Machinery, and some of these even contain additional subcategories such as Medical Equipment, Glassware, Stethoscopes, and others.

The nod link structure helps in identifying how categories are nested and how deeply certain branches go. Some branches are short and end some nodes, while others are more extensive,

showing a multi-level hierarchy. This layout is particularly useful in representing Xml based datasets where elements are organized in a tree-like structure. It provides a clear overview of relationships, allowing viewers to quickly understand the complexity and depth of the data. This visualisation technique is suitable for small to medium sized hierarchical datasets to get direct access to individual relationships. However, for very large datasets, node-link trees may become difficult and harder to interpret, especially when many nodes are expanded.

Radial Tree Layout

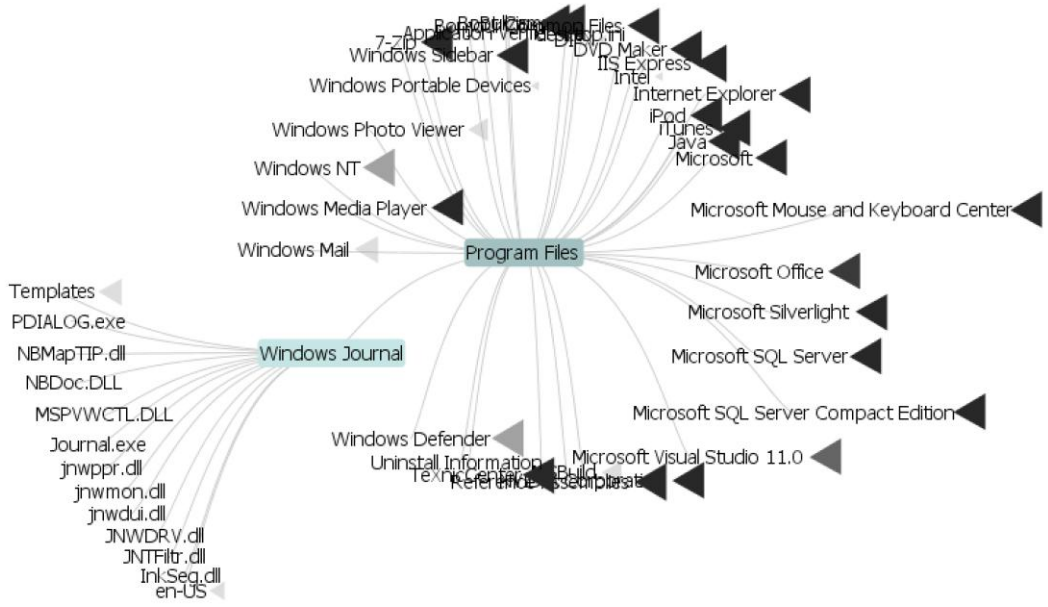

Figure 2

The second visualization uses a Radial Tree Layout to display hierarchical data in a circular fashion. In this diagram, the central node labeled “Program Files” represents the root of the hierarchy, and all other categories are extend outward radially like a circle or a wheel.
Each branch or segment represents a subfolder or program under the “Program Files” directory. Examples include Microsoft Office, Java, Windows Defender, Internet Explorer, etc. These are visualized with connecting lines that clearly indicate parent-child relationships between directories and subdirectories.
This radial layout is especially effective for:

- Providing a balanced view of hierarchical structures.

- Making complex data readable without long vertical or horizontal scrolling.
- Visually explaining the central node and how all elements stem from it.

This structure improves clarity for system administrators or users needing a quick overview of folder structures or installed applications. It also allows easy identification of clusters and isolated nodes.

Treemap Layout of Hierarchical Data

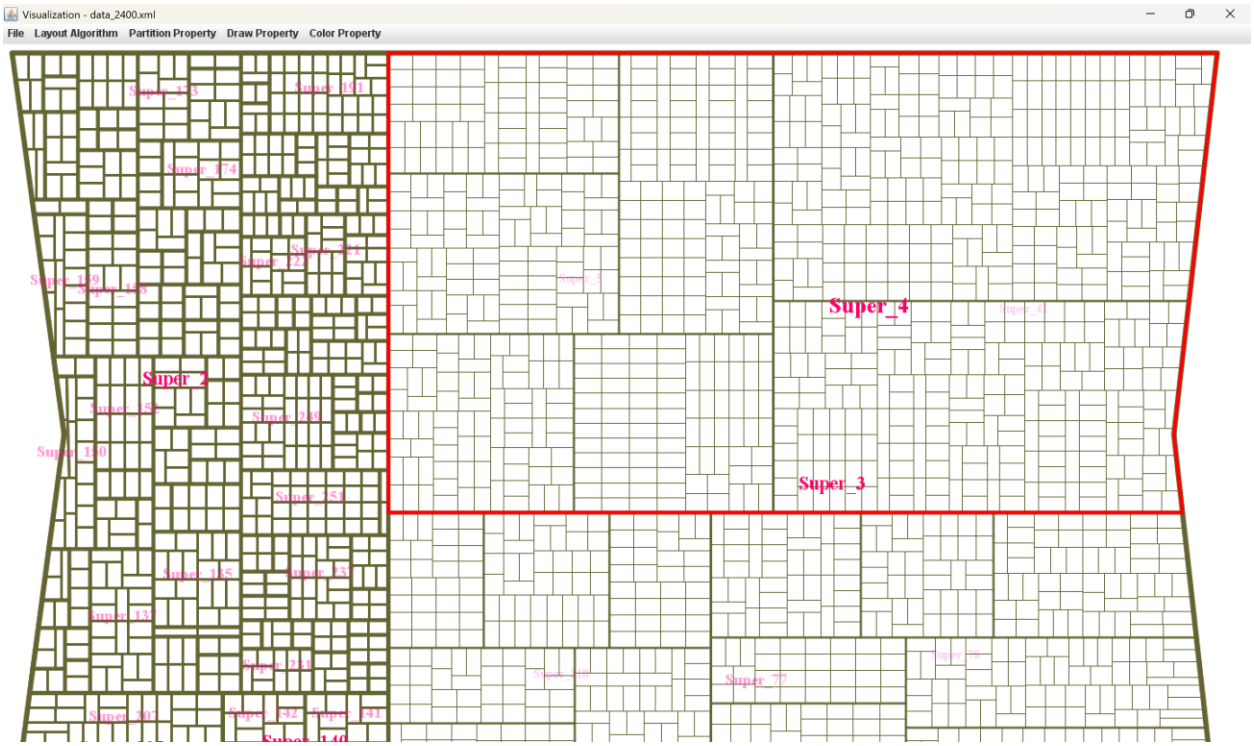

Figure 3

This treemap visualisation represents a hierarchical structure using a space-filling layout. Each rectangle denotes a node in the hierarchy, and its area is proportional to a specific attribute of the data, most likely a count or weight. The nested structure within each large rectangle reveals the parent-child relationship of the dataset.

In this specific treemap:

- The bold red boundaries highlight top-level categories like Super_2, Super_3, and Super_4.
- Each "Super" group is subdivided into smaller subgroups, shown as smaller rectangles.
- The nested levels indicate further depth in the hierarchy.
- Color (brown outlines) is used to separate node borders, helping to distinguish between them.

- The layout uses a slice-and-dice algorithm to maintain a compact and readable structure.

This visualization highlights the density and distribution of data within each top-level category such as Super_1, Super_2, and so on. It clearly shows how some groups contain many smaller child nodes, indicating a higher data density, while others are more aggregated with fewer, larger components. The treemap layout allows for an immediate visual comparison of both the size and structure of each group, helping to identify which sections are more complex or populated at a glance.

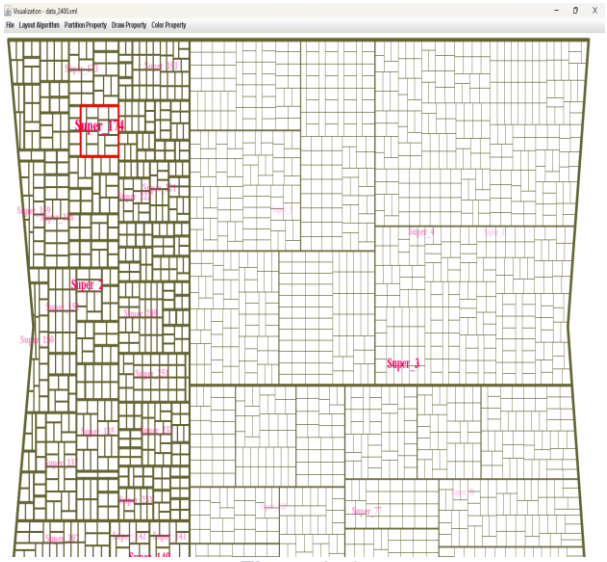

Figure 3. 1

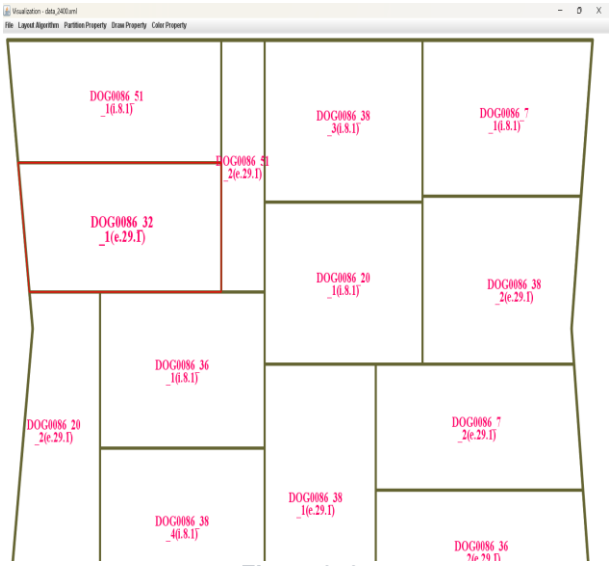

Figure 3. 2

Treemap Exploration of Super_174

The treemap visualization of Super_174 provides a focused view of the internal distribution within one specific top-level category. Each rectangle within the treemap represents a sub-node which is belonging to Super_174, where the size of each rectangle visually corresponds to its relative value or importance This focused drill-down makes it easier to analyze patterns and compare sub-groups within Super_174. For instance, we can immediately identify which sub-items occupy more space, suggesting higher relevance or volume. The treemap structure also reveals how well the data is clustered, and highlights differences in size, shape, and layout, offering insights into the category and also internal complexity.

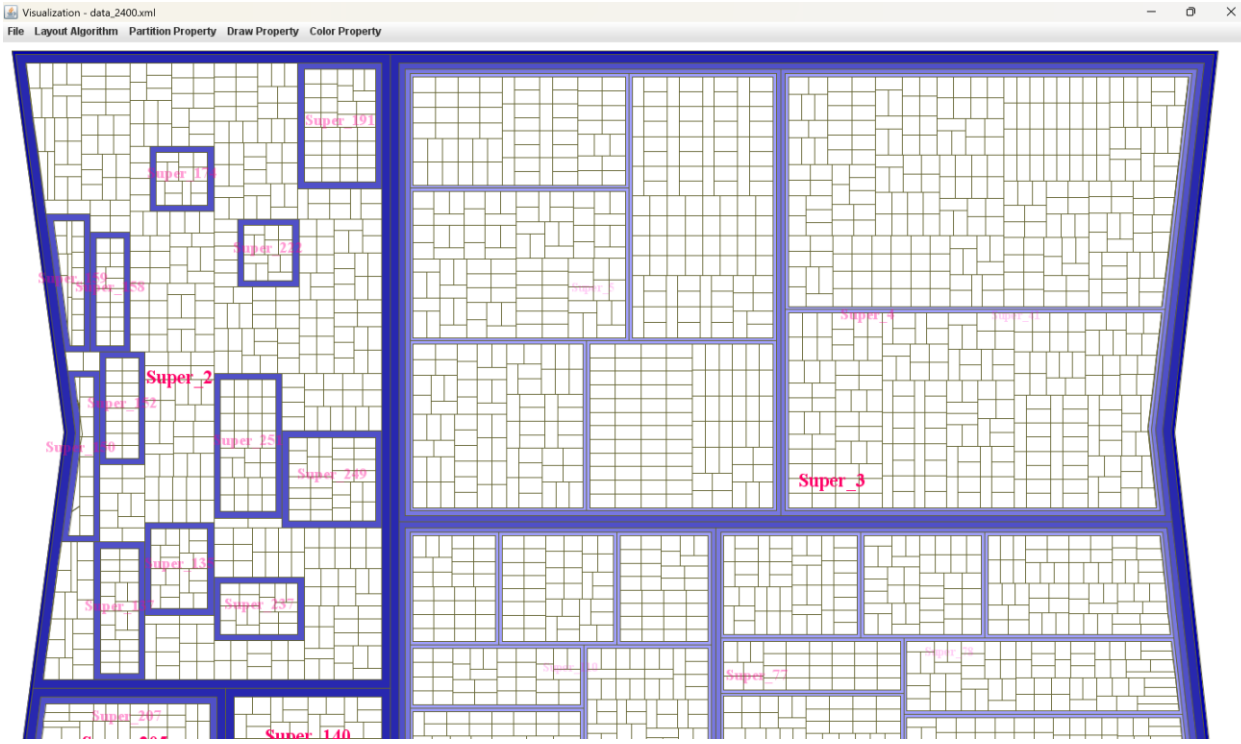

Figure 3. 3

To enhance the understanding of the treemap layout, boundary gaps were customized using the tool's Color Property > Boundary Color feature. Thicker and more vivid outlines (as seen in blue) were applied to visually separate major groupings.

This technique helps to emphasize group-level partitioning and makes it easier for viewers to quickly identify and compare the distribution of nested elements. While the tool doesn't automatically group by color, this manual customization allows for a more intuitive and structured view of the hierarchical data.

Space-Optimized Hierarchical Graph Visualisation

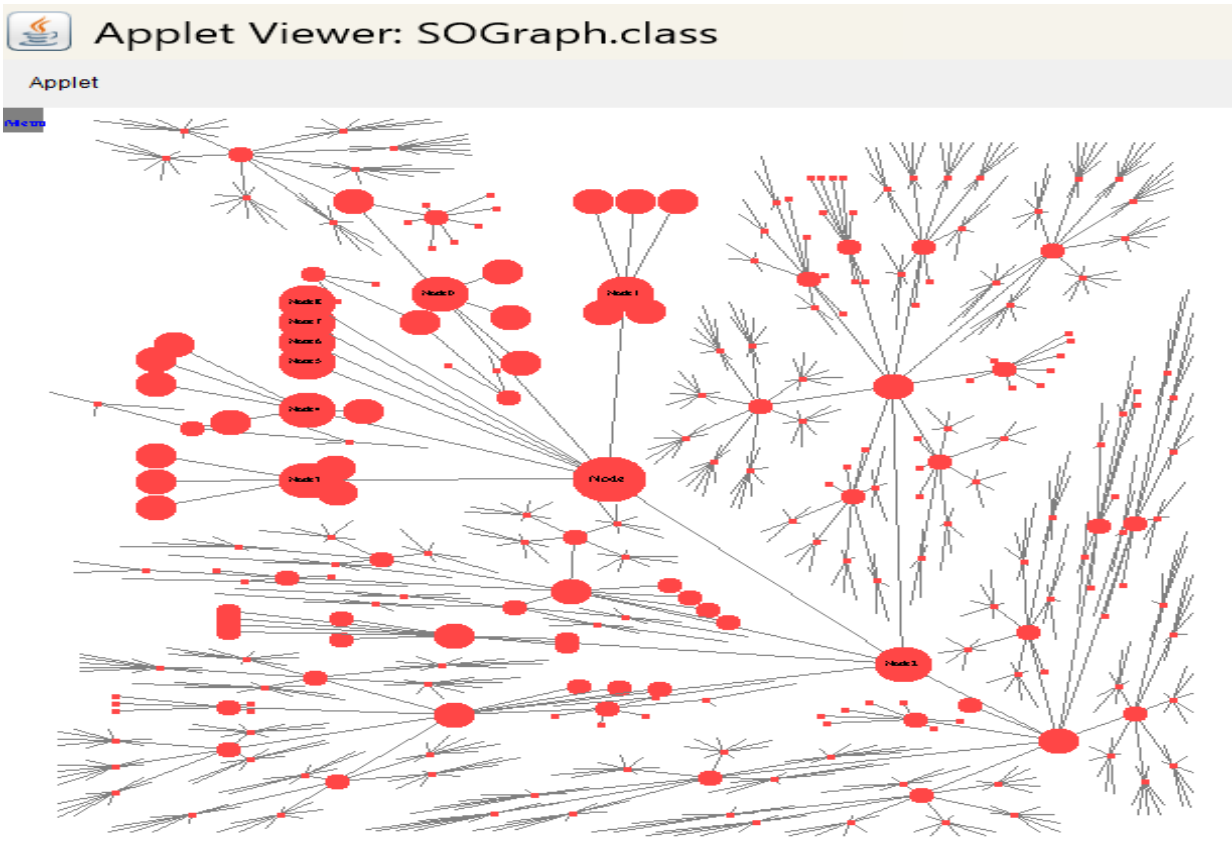

Figure 4

The above visualisation represents a hierarchical graph using a **space-optimized radial layout**. Each circle (node) represents an entity, and the lines (edges) show how they are connected in a parent-child structure.

In this layout, the root node is the most central or important node in the structure. All other nodes are arranged outward from it based on their levels in the hierarchy. This type of layout helps in saving space while still making the entire tree visible without overlapping too much.

Node sizes vary, indicating that some nodes may have more connections or importance than others. For example, Node 0 and Node 1 in the middle have many children and sub-branches, showing their high connectivity.

This layout is ideal when working with large graphs, as it uses space efficiently and avoids clutter. However, it may still become dense in the outer layers when the number of nodes grows very large.

Overall, this space-optimized layout is useful for quickly identifying the most influential nodes and understanding the depth and branching of the hierarchy.

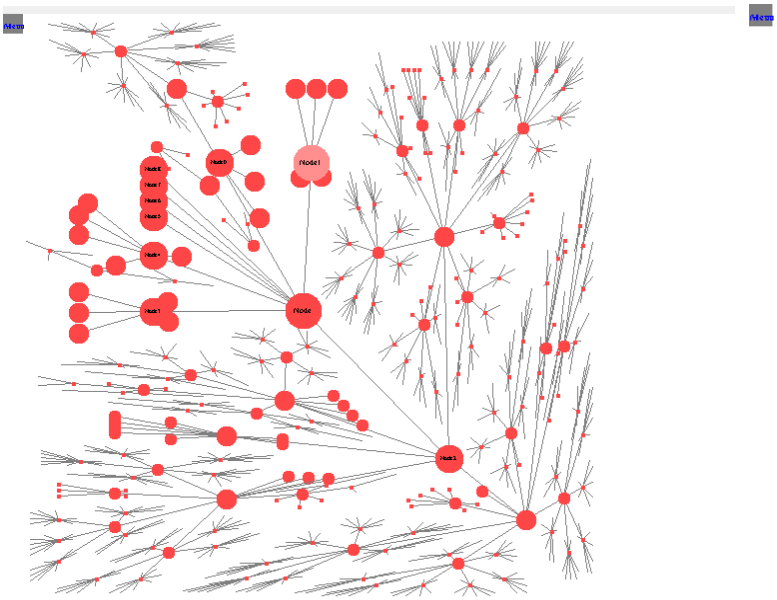

Figure 4. 1

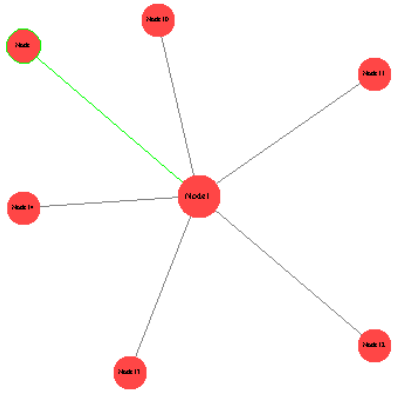

Figure 4. 2

When I selected Node 1, it got expanded. Now we can clearly see which nodes are directly connected to Node 1. This makes it easier to understand the structure and connections in the graph without showing everything at once. It helps focus on one part of the graph at a time.

DISCUSSION

Advantages and Disadvantages of the Visualisation Methods

Visualisation Method	Advantages	Disadvantages
Node-Link Tree Layout	- Easy to follow for small to medium hierarchies - Clearly shows parent-child relationships and paths	- Can become cluttered with large trees - Takes up more space as depth increases
Radial Tree Layout	- Space-efficient layout using circular form - Looks neat and balanced	- Hard to trace depth levels - Angled branches can make reading directions difficult
Treemap Visualisation	- Great for showing part-to-whole comparisons - Utilises screen space efficiently	- Relationships between nodes are not visually clear - Not ideal for deep hierarchies

Space-Optimised Hierarchical Graph	- Handles large datasets - effectively in compact space - Supports interactive expansion of nodes	- May confuse structural clarity if too many nodes are expanded - Can be harder to trace full paths

Effectiveness and Comparison of Visualisation Methods

The four visualisation techniques used Node-Link Tree Layout, Radial Tree Layout, Treemap Visualisation, and Space-Optimised Hierarchical Graph Visualisation are especially effective for visualising structured and the hierarchical data. Unlike force-directed graphs and adjacency matrices, which are better for general networks or dense connectivity patterns, these methods are tailored for tree-like or nested structures.

Node-link layouts clearly show parent-child relationships through connecting lines, which helps in tracing lineage and understanding hierarchy [5]. However, they may become cluttered with large datasets unless interactive navigation (like zooming or collapsing nodes) is used. Radial tree layouts, a variation of node-link layouts, distribute nodes in a circular pattern and are helpful when equal visual weight is preferred across branches. This radial approach has also been recognised in literature for improving balance and symmetry perception in visual exploration [2].

Treemaps, based on the enclosure approach, are ideal for the representation of the hierarchical data where the comparison of quantitative values have very much importance. They use nested rectangles to show part-to-whole relationships efficiently and are space-saving but may become visually complex with deep hierarchies [1].

Space-Optimised Hierarchical Graph Visualisation provides a practical way to display deeply nested hierarchies within a limited screen area. It supports scalability and readability, especially when paired with clustering or interactive filtering features [1].

Together, these four methods provide a strong visual toolout for representing large and complex hierarchical datasets, balancing clarity, structural insight, and spatial efficiency better than flat or purely relational visualisations.

CONCLUSION

The visualisation techniques such as Node-Link Tree Layout, Radial Tree Layout, Treemap, and Space-Optimised Hierarchical Graph. provides an unique advantages in visualising structured data. Node-Link and Radial Tree layouts make it easy to follow hierarchical relationships, especially when tracing from the root to leaf nodes. These are effective when working with

small to medium-sized datasets, but may become cluttered as the hierarchy deepens. Treemaps overcome space limitations by compressing data into rectangular regions, making it easier to compare node sizes and spot proportions at a glance. They are suitable when the focus is on visualising quantities rather than structure. The Space-Optimised Hierarchical Graph layout stands out in handling deep nesting with limited screen space. Its design minimises empty space and overlapping, and when paired with interaction such as node highlighting and expansion, users can explore complex hierarchies effectively.

Interactive Java-based visualisations help users better understand complex structures by making the layout and relationships clear and easy to follow [1]. Optimising the layout design also improves how easily users can read and scale large diagrams, which makes a big difference in handling complex data [3]. Focusing on the user's point of view and ensuring smooth flow of information is especially helpful when dealing with deeply nested or larger datasets [2]. Intuitive layouts and smart grouping of visual elements can also make it easier for people to make sense of complicated systems [5].

When it comes to handling large datasets, treemap visualisation has shown to be especially efficient by presenting a lot of information in a compact space without overwhelming the user [4]. It's particularly helpful for part-to-whole comparisons, where understanding proportions and sizes of elements is important [4].

Overall, the choice of visualisation method are mainly depending on the type of data and the main goal whether it's to find the paths, compare values, or explore large hierarchies interactively. These four techniques together show how using different layouts can bring out different insights from the same dataset, supported by ideas from the existing studies.

REFERENCES

1. Gestwicki, P. and Jayaraman, B., 2002, September. Interactive visualization of Java programs. In *Proceedings IEEE 2002 Symposia on Human Centric Computing Languages and Environments* (pp. 226-235). IEEE.

2. Andrews, K., Putz, W. and Nussbaumer, A., 2007, July. The hierarchical visualisation system (HVS). In *2007 11th International Conference Information Visualization (IV'07)* (pp. 257-262). IEEE.

3. Lambert, A., Queyroi, F. and Bourqui, R., 2012, July. Visualizing patterns in node-link diagrams. In *2012 16th International Conference on Information Visualisation* (pp. 48-53). IEEE.

4. Jadeja, M. and Shah, K., 2015. Tree-Map: A Visualization Tool for Large Data. *GSB@ SIGIR*, *1393*, pp.p-9.

5. Hamer, J., 2004. Visualising java data structures as graphs. *ACE*, *4*, pp.125-129.